

NLP Assignment No. 1 — Answers

Subject Code: BCSE-573 | Subject: Natural Language Processing

Section – A (1 mark each)

i. How is a text file opened in both read and write mode?

In Python, files are opened with the built-in `open()` function, which takes a filename and a mode string.

- Read mode: `open("data.txt", "r")` — opens the file only for reading; raises an error if the file does not exist.
- Write mode: `open("data.txt", "w")` — opens the file for writing; creates it if absent, and overwrites (truncates) it if it exists.
- Read + write mode: `open("data.txt", "r+")` opens an existing file for both reading and writing without erasing its content. `open("data.txt", "w+")` also allows both, but truncates the file first.

The file should be closed after use with `close()`, or opened using a context manager so it closes automatically:

```
with open("data.txt", "r+") as f:
    content = f.read()
    f.write("\nAppended line")
```

ii. Differentiate between spaCy and NLTK.

Aspect	NLTK	spaCy
Purpose	Research and teaching toolkit	Production-ready NLP library
Speed	Slower (pure Python)	Fast (built in Cython)
Design	Offers many alternative algorithms to choose from	Provides one optimized pipeline per task
Ease of use	More code required, string-based	Simple API: <code>nlp(text)</code> returns rich Doc object
Pretrained models	Limited built-in statistical models	Ships pretrained pipelines for POS, NER, parsing
Typical use	Learning NLP concepts, prototyping	Building real-world NLP applications

iii. Output of: `re.findall(r"\d$", 'These ends with a number 203')`

The pattern `\d$` matches a single digit only when it occurs at the very end of the string. Since the string ends in "203", only the last character ("3") satisfies the anchor.

```
Output: ['3']
```

iv. How to get the sentences of a text document using spaCy?

spaCy's Doc object exposes a `doc.sents` generator (based on the dependency parser) that yields Span objects, one per sentence:

```
import spacy
nlp = spacy.load("en_core_web_sm")
```

```
doc = nlp("This is sentence one. This is sentence two.")

for sent in doc.sents:
    print(sent.text)
```

v. Examples of use cases of NLP.

- Machine translation (e.g., Google Translate)
- Sentiment analysis of reviews and social media
- Chatbots and virtual assistants (Siri, Alexa)
- Spam email detection
- Speech recognition
- Text summarization
- Autocomplete and grammar/spell correction
- Question answering systems

vi. Shorthand character classes with example.

Regular expressions provide shorthand codes for common character sets:

Class	Meaning
\d	Any digit (0-9)
\D	Any non-digit character
\w	Any word character (letters, digits, underscore)
\W	Any non-word character
\s	Any whitespace character (space, tab, newline)
\S	Any non-whitespace character
<pre>import re re.findall(r"\d+", "Room 42 has 10 chairs") # Output: ['42', '10']</pre>	

Section – B (2 marks each)

Q.2. Differentiate between stemming and lemmatization with an example.

Aspect	Stemming	Lemmatization
Approach	Crude, rule-based suffix chopping	Dictionary/morphology based lookup
Output	May not be a valid word (a stem)	Always a valid dictionary word (a lemma)
Context awareness	No	Yes (can use POS tag)
Speed	Faster	Slower (more processing)
Example	"studies" -> "studi", "running" -> "run"	"studies" -> "study", "better" -> "good"

```

from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer

stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

print(stemmer.stem("studies"))      # studi
print(lemmatizer.lemmatize("studies")) # study

```

Q.3. Explain the role of deep learning in NLP.

Traditional NLP relied on hand-crafted features and rules, which struggled to generalize. Deep learning automates feature extraction directly from raw text and has driven most recent progress in NLP:

- Word embeddings (Word2Vec, GloVe) represent words as dense vectors that capture semantic relationships.
- RNNs/LSTMs/GRUs model sequential, order-dependent data such as sentences, useful for translation and sentiment analysis.
- Transformer architectures (BERT, GPT) use self-attention to build context-aware representations, achieving state-of-the-art results in translation, summarization, and question answering.
- Transfer learning: large pretrained language models can be fine-tuned on small task-specific datasets, reducing the need for labeled data.
- End-to-end learning removes the need for separate hand-built pipelines for tokenization, features, and classification.

Applications enabled by deep learning include chatbots, machine translation, text summarization, speech recognition, and text generation.

Section – C (4 marks each)

Q.4. Define Tokenization. Retrieving sentences, tokens, noun chunks, and entities with spaCy.

Tokenization is the process of breaking a stream of text into smaller meaningful units called tokens — typically words, punctuation marks, or subwords — which serve as the basic input units for further NLP processing.

```

import spacy
nlp = spacy.load("en_core_web_sm")
doc = nlp("Apple is looking at buying U.K. startup for $1 billion. Tim Cook made the announcement.")

# 1. Sentences
for sent in doc.sents:
    print(sent.text)

# 2. Tokens
for token in doc:
    print(token.text)

# 3. Noun chunks
for chunk in doc.noun_chunks:
    print(chunk.text)

# 4. Named entities
for ent in doc.ents:
    print(ent.text, ent.label_)

```

doc.sents splits the text into sentences using the parser; iterating over doc gives individual tokens; doc.noun_chunks yields base noun phrases (e.g., "Apple", "U.K. startup"); and doc.ents yields named entities with their labels (e.g., "Apple" -> ORG, "\$1 billion" -> MONEY, "Tim Cook" -> PERSON).

Q.5. Program to copy a single page from a PDF and append it to another PDF.

```
from pypdf import PdfReader, PdfWriter

def copy_page(source_path, page_number, dest_path, output_path):
    # page_number is 0-indexed
    source_reader = PdfReader(source_path)
    page_to_copy = source_reader.pages[page_number]

    dest_reader = PdfReader(dest_path)
    writer = PdfWriter()

    # Copy all existing pages of the destination PDF
    for page in dest_reader.pages:
        writer.add_page(page)

    # Append the page copied from the source PDF
    writer.add_page(page_to_copy)

    with open(output_path, "wb") as f:
        writer.write(f)

# Example: copy page 3 (index 2) of source.pdf onto the end of
# destination.pdf
copy_page("source.pdf", 2, "destination.pdf", "merged_output.pdf")
```

PdfReader loads each PDF and exposes its pages as an indexable list; PdfWriter is used to assemble a new PDF by adding the destination file's pages followed by the extracted source page, then writing the result to a new output file.

Section – D (6 marks each)

Q.6. Define Stop Words. Adding, removing, and checking stop words.

Stop words are common words in a language (e.g., "the", "is", "at", "which", "on") that occur very frequently but carry little distinctive meaning. They are often filtered out before tasks such as search indexing, text classification, or keyword extraction to reduce noise and focus on meaningful content.

i) Adding a word to the default stop word set:

```
import spacy
nlp = spacy.load("en_core_web_sm")

nlp.Defaults.stop_words.add("btw")
nlp.vocab["btw"].is_stop = True
```

ii) Removing a word from the default stop word set:

```
nlp.Defaults.stop_words.remove("btw")
nlp.vocab["btw"].is_stop = False
```

iii) Checking whether a word is a stop word:

```
print(nlp.vocab["the"].is_stop)    # True
```

```
print(nlp.vocab["hello"].is_stop)    # False
```

Filtering stop words out of a full sentence:

```
doc = nlp("This is a sample sentence showing stop word filtering")
filtered_tokens = [token.text for token in doc if not token.is_stop]
print(filtered_tokens)
# ['sample', 'sentence', 'showing', 'stop', 'word', 'filtering']
```

Q.7. Quantifiers and identifiers used in regular expressions.

Quantifiers specify how many times a preceding element may occur:

Quantifier	Meaning	Example
*	0 or more occurrences	ab* matches "a", "ab", "abb"
+	1 or more occurrences	ab+ matches "ab", "abb" (not "a")
?	0 or 1 occurrence (optional)	colou?r matches "color" and "colour"
{n}	Exactly n occurrences	\d{3} matches exactly 3 digits
{n,}	n or more occurrences	\d{2,} matches 2 or more digits
{n,m}	Between n and m occurrences	\d{2,4} matches 2 to 4 digits

Identifiers (anchors and boundaries) specify a position in the text rather than matching a character:

Identifier	Meaning	Example
^	Start of the string/line	^Hello matches "Hello" only at the start
\$	End of the string/line	end\$ matches "end" only at the end
\b	Word boundary	\bcata\b matches the whole word "cat", not "category"
\B	Non-word boundary	\Bcat matches "cat" inside "concatenate"

```
import re
re.findall(r"\bcata\b", "cat category concatenate cat")
# Output: ['cat', 'cat']
```